CRDs & Operators

Kubernetes Production · Lesson 15

Built from the official Kubernetes documentation

`kubernetes.io/docs`

Learning Objectives

By the end of this lesson you will be able to:

- Explain that the Kubernetes API is **declarative** and **extensible** (**CRI / CNI / CSI**)
- Define a **custom resource** and when to use one vs a ConfigMap
- **Read** a **CustomResourceDefinition** and use its custom resources (`get` / `explain`)
- Describe the **controller control loop** and the **Operator pattern**
- **Operate** an operator: install it, drive it via CRs, read its `.status`
- Manage the CR lifecycle: **ownerReferences / GC** and **finalizers** (stuck deletions)
- **Troubleshoot** a misbehaving operator (controller logs, webhooks, RBAC)

> Reference: *Extending Kubernetes; Custom Resources; Operator pattern* — kubernetes.io/docs

15.1 Extending Kubernetes

A Declarative, Extensible API

Kubernetes is built around a **declarative API**: you describe **desired state** in objects, and **controllers** continuously reconcile reality to match it.

The platform is designed to be **extended** without forking Kubernetes:

- **Client extensions** — `kubectl` / credential plugins
- **API-access** — authentication, authorization, **admission webhooks**
- **API additions** — **CRDs** and the **aggregation layer**
- **Infrastructure** — **CRI**, **CNI**, **CSI**, device plugins
- **Scheduling** — custom schedulers / extenders
- **Controllers** — watch state and reconcile

> This lesson focuses on the **API-additions** point: **custom resources**.

The Extension Points

Extension point	What it customises
kubectl / credential plugins	Client subcommands, auth
Authn / Authz	Who calls the API, and what they may do
Admission webhooks	Block, mutate, or validate requests
CRDs / aggregation layer	Add new API kinds
CRI	Container **runtime** interface
CNI	Pod **network** interface
CSI	Container **storage** interface
Scheduling / controllers	Placement and reconciliation

> The **webhook** and **controller** patterns recur across many of these points.

Infrastructure Interfaces: CRI / CNI / CSI

The CKA expects you to **recognise** the three infrastructure interfaces — stable APIs that let a cluster swap implementations:

- **CRI — Container Runtime Interface**: how the **kubelet** talks to the container runtime (e.g. **containerd**, **CRI-O**)
- **CNI — Container Network Interface**: pluggable **Pod networking** (e.g. **Calico**, **Cilium**, **Flannel**)
- **CSI — Container Storage Interface**: pluggable **storage backends** mounted into Pods

> Each is an **interface** behind which vendors provide plugins — the control plane stays the same while runtime / network / storage are swapped.

15.2 Custom Resources

What Is a Custom Resource?

A **custom resource** is *"an extension of the Kubernetes API that is not necessarily available in a default Kubernetes installation."*

- It is an **endpoint in the Kubernetes API** that stores a collection of objects
- It can **appear and disappear dynamically** in a running cluster
- You use it with `kubectl` **just like built-in resources** (Pods, Deployments)

> On its own, a custom resource only **stores structured data**. Combined with a **custom controller**, it becomes a true **declarative API** that *acts*.

- Caution: don't use a custom resource as **data storage** for application or end-user data — that couples your app too tightly to the API server.

ConfigMap vs Custom Resource

Use a ConfigMap when…	Use a Custom Resource when…
There is an existing config-file format (e.g. `mysql.cnf`)	You want top-level `kubectl get <kind>` support
A program in a Pod reads the file to configure itself	You want API conventions: `.spec`, `.status`, `.metadata`
Consumers prefer file/env access	You want automation that **watches** and reconciles
Rolling updates come from the Deployment	The object abstracts a collection of controlled resources

> Reach for a **custom resource** when the object is a first-class **API** that something should act on — not just a config blob.

15.3 Two Ways to Add a Custom API

CRD vs API Aggregation

	CustomResourceDefinition	API Aggregation
Programming	**None** — declarative YAML	**Required** — build an apiserver
What runs	The existing kube-apiserver	A **subordinate apiserver** behind a proxy
Flexibility	Less (generic impl.)	More — storage, version conversion
Typical use	The **common path**	Complex APIs needing fine control

> *"Kubernetes provides these two options … so that neither ease of use nor flexibility is compromised."*

- **CRD is the no-code, common path** — and the focus of the CKA.

15.4 CustomResourceDefinition

The CRD — the essentials

A **CustomResourceDefinition** registers a **new kind**; the apiserver then serves it like any built-in.

- `apiVersion: apiextensions.k8s.io/v1` · `kind: CustomResourceDefinition`
- `metadata.name` **must** be `<plural>.<group>` (`crontabs.stable.example.com`)
- `spec.scope`: `Namespaced` or `Cluster` — but **the CRD object is always cluster-scoped**
- `spec.versions[]`: each is `served` (reachable) + **exactly one** `storage: true`
- `spec.names`: `plural` / `singular` / `kind` (CamelCase) / `shortNames`
- `spec.versions[].schema.openAPIV3Schema`: a **structural schema** (mandatory in v1)

> As an admin you mostly **receive** CRDs (installed by an operator) — you rarely hand-write the schema. Knowing the fields is enough to **read and debug** one.

A CRD by example (CronTab)

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: crontabs.stable.example.com     # <plural>.<group>
spec:
  group: stable.example.com
  scope: Namespaced
  names: { plural: crontabs, singular: crontab, kind: CronTab, shortNames: ["ct"] }
  versions:
  - name: v1
    served: true
    storage: true                        # exactly one storage version
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            properties: { cronSpec: {type: string}, image: {type: string} }
```

Optional enrichments: **`additionalPrinterColumns`** (extra `kubectl get` columns) and **subresources** — `status` (controller writes status) and `scale` (lets `kubectl scale` / the **HPA** drive the CR).

Create & discover custom resources

A custom resource is just YAML with `<group>/<version>` + the `kind`:

```yaml
apiVersion: stable.example.com/v1
kind: CronTab
metadata: { name: my-cron }
spec: { cronSpec: "*/5 * * * *", image: my-cron-image }
```

The **admin's discovery workflow** on an unfamiliar cluster:

```
kubectl get crd                       # what extensions are installed?
kubectl api-resources | grep stable   # which kinds does this group add?
kubectl explain crontab.spec          # what fields does the kind accept?
kubectl get crontabs -A               # the actual objects, all namespaces
```

> CKA flow: **`get crd`** → **`api-resources`** → **`explain`** — read any custom API you land on without prior knowledge.

15.5 Controllers & Operators

The Controller Control Loop

Every Kubernetes controller runs a **control loop**:

1. **Observe** — read the object's `.spec` (desired state)
2. **Diff** — compare desired state with **actual** cluster state
3. **Act** — make changes to reconcile, then update `.status`

> This loop is the **core pattern** of Kubernetes — built-ins and custom controllers work the same way.

The Operator Pattern

Operators are *"software extensions to Kubernetes that make use of custom resources to manage applications and their components"* and *"follow Kubernetes principles, notably the control loop."*

Operator = CRD(s) + a custom controller

- The **CRD** defines the API (e.g. a `SampleDB` kind)
- The **controller** runs the loop and **encodes operational knowledge** — how a human operator would deploy, back up, upgrade and recover the app
- The controller usually runs **outside the control plane**, as a **Deployment**

> You declare *what you want* (a custom resource); the operator makes reality match and keeps it healthy.

What Operators Automate

Operators capture the expertise of a human operator. From the docs, they can:

- **Deploy** an application on demand
- **Take and restore backups** of application state
- **Handle upgrades** — including schema and config changes
- **Publish services** for apps that don't speak the Kubernetes API
- **Simulate failure** to test resilience
- Manage **leader election** for distributed apps

> **Real examples**: **Prometheus Operator**, **etcd** / database operators (PostgreSQL, MySQL), **cert-manager**.

Installing an Operator

*"The most common way to deploy an operator is to add the **CRD** and its **Controller** to the cluster."* Via **manifests**, a **Helm chart** (see Lesson 19), or **OLM / OperatorHub** — installation brings in:

- the **CRDs** (the new kinds)
- the **controller Deployment** (runs the reconcile loop, usually in its own namespace)
- the **RBAC** it needs (ServiceAccount + ClusterRole/Role bindings)

> **OLM (Operator Lifecycle Manager)** + **OperatorHub.io** are how operators are catalogued, installed and upgraded in production — a *package manager for operators*.

Operating an operator, day-to-day

You don't touch the objects it manages — you **edit the custom resource** and let it reconcile:

```
kubectl get sampledb -A                       # the instances it manages
kubectl describe sampledb/example -n prod     # spec + status + Events
kubectl edit sampledb/example -n prod         # change desired state → it reconciles
kubectl get sampledb/example -o jsonpath='{.status}'   # what the controller reports
```

> 🔑 **Golden rule:** change the **CR**, never the Deployments/Services/PVCs the operator created — the control loop will just revert them to match the CR.

ownerReferences & garbage collection

An operator stamps `ownerReferences` on everything it creates, pointing back to the CR — this wires up **cascade deletion**:

```
metadata:
  ownerReferences:
  - apiVersion: demo.example.com/v1
    kind: Widget
    name: w1
    uid: <widget-uid>          # the owner's UID
```

- Delete the **CR** → Kubernetes **garbage-collects** every owned object automatically.
- Cascade modes: **background** (default), **foreground** (`--cascade=foreground`), **orphan** (`--cascade=orphan`, keep the children).

> This is why deleting one custom resource can tear down a whole app — the operator **owns** the pieces.

Finalizers & stuck deletions (the classic incident)

A **finalizer** is a string that **blocks deletion** until a controller runs cleanup (drain, deregister, delete cloud resources) and removes it:

```
$ kubectl delete widget w2       # hangs — the object goes to "Terminating"
$ kubectl get widget w2 -o jsonpath='{.metadata.deletionTimestamp} {.metadata.finalizers}'
2026-...Z ["demo.example.com/cleanup"]      # marked for deletion, but still there
```

⚠️ If the operator is **down or uninstalled**, nothing removes the finalizer → the CR (or its **namespace**) is **stuck** `Terminating` **forever**. Last-resort unblock:

```
kubectl patch widget w2 --type=merge -p '{"metadata":{"finalizers":null}}'
```

> Prefer fixing the controller so cleanup actually runs; strip the finalizer **only** when the operator is gone for good.

Troubleshooting operators

Symptom	Where to look
CR ignored, empty `.status`	the controller **Deployment** — Running? `kubectl logs` it
Every CR create/edit fails	an **admission / conversion webhook** is down (operator offline)
CR stuck `Terminating`	a **finalizer** + a dead controller (previous slide)
`Forbidden` in controller logs	the operator's **RBAC** lacks a verb/resource (Lesson 14)

```
kubectl -n <op-ns> logs deploy/<operator-controller> --tail=50
kubectl describe <kind>/<name>            # Events explain most reconcile failures
kubectl get crd <name> -o yaml | grep -A5 conditions
```

> Reconcile problems are **almost always** in the controller's logs or the CR's **Events** — start there, not in the managed objects.

Key Takeaways

- The Kubernetes API is **declarative** and **extensible**; **CRI / CNI / CSI** are the infrastructure interfaces (runtime / network / storage)
- A **custom resource** extends the API (used with `kubectl` like built-ins); alone it stores data — a **controller** makes it act. CRD = `apiextensions.k8s.io/v1`, named `<plural>.<group>`, **cluster-scoped**, **one** `storage` version
- **Operator = CRD(s) + controller** running **observe → diff → act**; installing brings **CRDs + Deployment + RBAC** (manifests / Helm / **OLM**)
- **Operate by editing the CR**, never the managed objects — they get reverted
- Operators wire **ownerReferences** (cascade **GC**) and **finalizers** (cleanup); a dead controller + finalizer = **stuck** `Terminating` → patch finalizers as last resort
- **Debug** via the controller's **logs** and the CR's **Events**

End of Lesson 15

Next: Lesson 16 — Node Maintenance & Scaling

`kubectl get crd` · `kubectl api-resources` · `kubectl explain <kind>` · `kubectl apply -f crd.yaml`